Nearest Neighbour Classification

Template Matching

- each class represented by one template (prototype)
- classified objects are templates with added noise
- comparing objects to templates by maximum corelation or minimum error

Minimum-distance classification

- **feature vector** in place of template:
  - $t_1, t_2, ..., t_L$
  - L - number of classes in problem
- classification:
  - compute feature vector of object x
  - then we compute L distances between x and each template
  - finaly we find minimum distance between x and class templates

Distance functions

- usualy written as $d(x, y)$
- must satisfy 4 axioms:
  - **reflexivity** - if $d(x, y) = 0$ then $x = y$
  - **non-negativity** - $d(x, y) \geq 0$
  - **symmetry** - $d(x, y) = d(y, x)$
  - **triangle inequality** - $d(x, y) + d(y, z) \geq d(x, z)$

1-NN

- nearest neighbour classification
- we try to find class that is nearest to our classified object
- only one nearest neighbour decides about the classification result

Training set

- training set: $T = \{\langle x^k, y^k \rangle, k = 1, 2, ..., N\}$
- pairs of feature vectors x^k and their class labels y^k
- we find minimal distance between our classified object and feature vectors from training data

Advantages

- direct use of training set
- low probability of error
- simple implementation

Disadvantages

- large computational complexity for large N
- sensitive to misclassified samples
- random behaviour at class boundaries

k-NN

- majority voting of k neares neighbours
- we get a set of neighbours T^k from training set T

$$T^k = \{\langle x^v, y^v \rangle, v = 1, 2, ..., k\}$$

- we take a subset of T^k which contains elements of class y ($T^{k,y}$)

- membership function $F^{y(x)}$ - we count how many of k nearest neighbours of class x belongs to class y
- problem - we need k both to be:
  - large - to minimize misclassification
  - small - to seperate classes strongly
- we use training data directly so classifier needs to store all training vectors and their class labels:
  - N - number of samples
  - D - number of features
  - we need memory of $N \times D + N = N(D + 1)$
  - k parameter is irrelevent

Data preprocessing

- in NNC it's important that all features should be equally important
- first step of designing classifier: learn the data
  - detect outliers
  - make decision if normalization is needed

Normalization

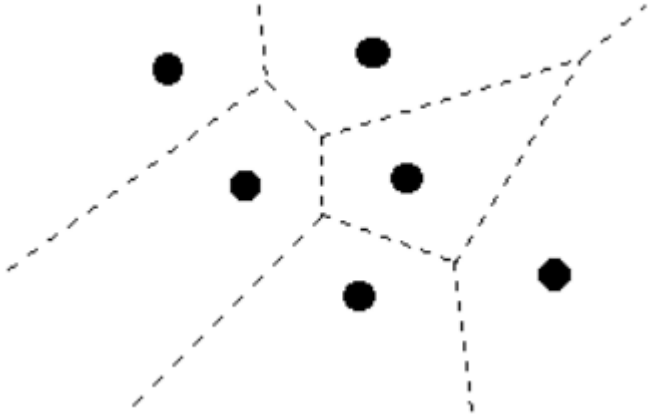
- usually simple operations will do (mean and variance of features, min-max values, plotting histogram)
- in many cases of normalization, scaling the input data will do
  - if you normalize training set, data to classify should be normalized in the exact same way
- normalizing data according to their distribution have some pros:
  - features become comparable (one does not dominate over another)
  - numeric stability
  - often improves performance of classifiers, such as (k-NN, k-means, SVM)
- but it has flawes:
  - some features **are** more important than others
  - scaling is sensitive to outliers
  - again, you have to scale test and inputed data

HNSW

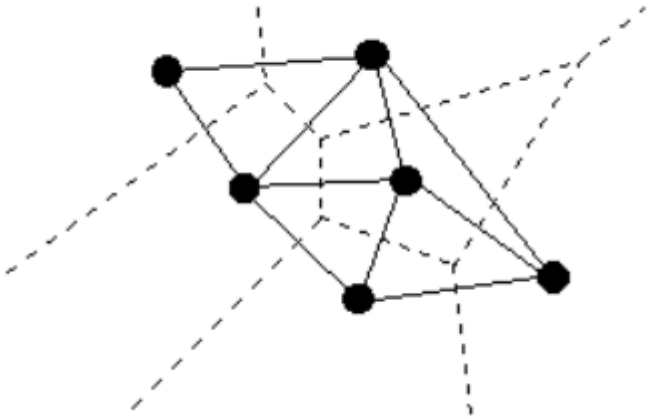
- multi-level map
- top layer has the fewest points
- botton has all points
- we find nearest neighbour to the query, then we go layer down
- you need to build it first, and it's not always 100% correct
- advantages:
  - at first you jump very far
  - then zoom in precisely
  - logarithmic time instead of linear

Voronoi diagram

- for given set of nodes, you divide space into regions in which all points are closer to region's node than any other node



- **Delaunay triangulation** - if two Voronoi regions have common edge, you connect the nodes of those regions
  - they become **Voronoi/Delaunay neighbours**:



Condensing algorithm

1. Compute Delaunay triangulation for training set
2. For each node, mark, if all it's Delaunay neighbours belong to the same class as the node considered
3. Remove all marked nodes (space for chosen node expands). Remaining nodes comprise condensed training set

Editing algorithm

1. Build proximity graph of the training set
2. For each node:
   - Find all it's neighbours
   - Determine most frequent class in the neighbourhood
   - Mark node if it belongs to other class
3. Remove marked nodes